KAVACH

The complete guide

A local shield that stops personal data from leaking into AI chatbots

How it was built · how every part works · why each technology was chosen · why it is a strong use of light open-weight models · file-by-file reference · pitch and Q&A

Hacktoberfest Hack Day Pune · Track 1: Best Open-Source AI Project · PS#3: PII redaction for Indian documents and languages
Repository: github.com/diipanshuuuu/Kavach · License: MIT

Contents

#	Section
1	Kavach in one page
2	The build journey: how we got here
3	Architecture: the big picture
4	Detection in depth: rules, NER, LLM, merge
5	Files in depth: images, PDFs, PowerPoint, QR codes, faces
6	The browser extension in depth
7	Manual hiding
8	Security and privacy design
9	The one-step installer
10	Evaluation: how we measured it
11	Why we chose each technology
12	Why this is a strong use of light open-weight models
13	File-by-file reference
14	Models, tools and licenses
15	Honest limitations and what comes next
16	4-minute pitch script
17	Likely judge questions and answers
18	Glossary

1. Kavach in one page

The problem

Every day people paste their Aadhaar number, PAN card, bank statement, boarding pass or medical report into ChatGPT, Gemini or Claude to get help: "fill this form", "explain this bill", "summarise this report". The moment they press paste or upload, that personal data is on someone else's servers. It may be logged, reviewed by people or used for training, and it can never be called back.

Existing redaction tools are built for English and US formats (SSNs, US phone numbers). They don't know what an Aadhaar, PAN, GSTIN, IFSC or UPI ID looks like, and they don't handle Hindi or Marathi text, scanned Indian ID cards or boarding-pass barcodes. Many of them also send your document to their own cloud to find the PII, which defeats the purpose.

The solution

Kavach ("armour") is a browser extension plus a small engine that runs on your own laptop.

1. It **catches** the paste or upload on the chatbot's page *before* the website sees it.
2. It **finds** the personal data locally, using checksum-backed rules, an open-weight NER model and, optionally, a small open-weight LLM. For files it also uses OCR and finds faces, QR codes and barcodes.
3. It **shows a review card**. The user sees exactly what will be hidden, can untick items, and can hide anything extra.
4. It **masks** each item with a placeholder like `[PERSON_1]` or `[AADHAAR_1]`. Files are redacted with black boxes, and codes and faces are blurred.
5. It **restores** the real values on your own screen when the chatbot's reply mentions `[PERSON_1]`, so the conversation still reads naturally.

> **The core promise:** the chatbot company only ever receives placeholders. The real values never leave the laptop. They live in a session-only vault in the browser, and no part of Kavach runs in the cloud.

What it handles

Input	How Kavach handles it
Pasted text	Detected, reviewed, masked with placeholders, restored in replies
Dragged or picked images (PNG/JPG)	OCR (English, Hindi, Marathi); PII boxed in black; faces and QR/barcodes blurred
PDFs (digital or scanned)	Text layer used if present, OCR if scanned; rebuilt from images so no hidden text survives
PowerPoint (PPTX)	Text replaced in place; pictures inside slides redacted; file metadata scrubbed
Anything detection missed	Manual hiding: click a word, type a phrase, draw a box, or keep an "always hide" list

Key numbers

Measure	Value
Recall with rules + NER (default)	97.3% on 150 synthetic documents
Precision with rules + NER	100% (zero false alarms, including deliberate decoys)

Measure	Value
Time per document (default)	~58 ms on a laptop CPU, no GPU
With optional Qwen3-4B deep scan	99.3% recall, 86% precision, ~4 s per document
Data sent to any cloud by Kavach	None
Setup on a new laptop	One command: ./install.sh

2. The build journey: how we got here

Kavach was built in layers. Each step solved a concrete gap found in the previous one. This is the order it happened in, and the reason behind each step.

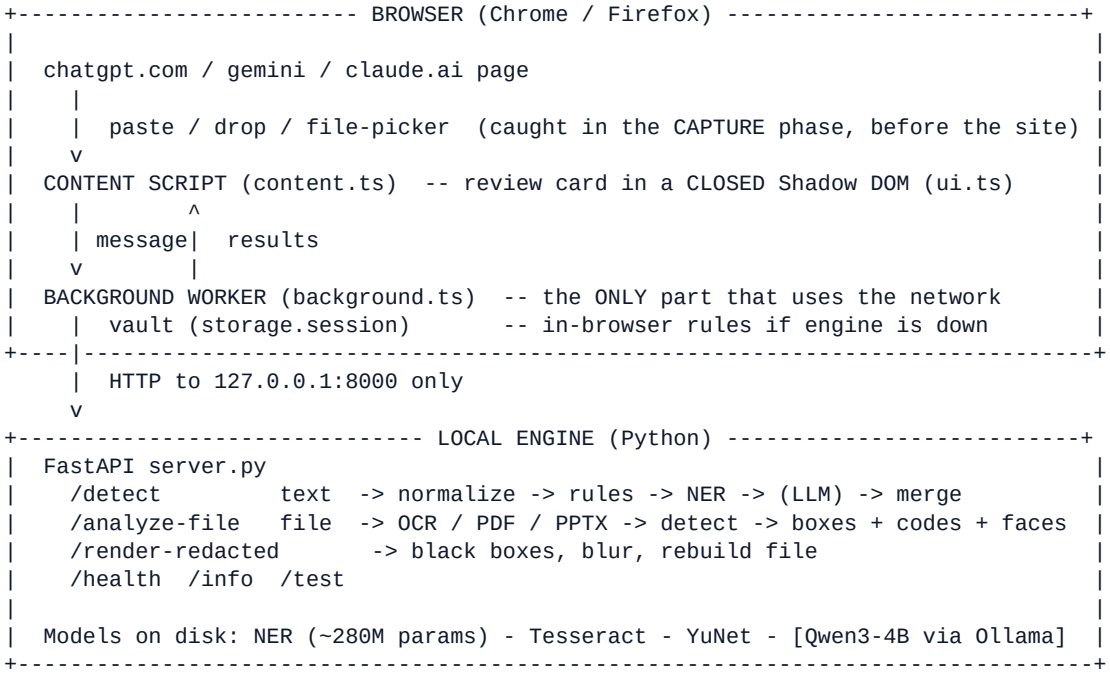
#	Phase	What we built	Why
1	Planning	Chose PS#3 and Track 1. Chose a browser extension over a separate app or website.	The leak happens inside the browser at the moment of pasting. An extension can stop it right there without changing how people work.
2	Engine skeleton	A FastAPI server listening only on 127.0.0.1:8000, with typed request and response schemas.	Python is where the AI, OCR and PDF tools are. Binding to 127.0.0.1 means nothing outside this laptop can reach it.
3	Layer 1: rules + checksums	Patterns for Aadhaar, VID, PAN, GSTIN, IFSC, UPI, phone, email, card, bank account, DOB, PIN code, passport, voter ID, driving licence, vehicle number. Each is validated mathematically where possible.	Indian IDs carry check digits, so maths can prove a number is real. That gave 100% precision with no AI.
4	Layer 2: NER model	Open-weight named-entity model (IndicNER first, automatic fallback to XLM-RoBERTa WikiANN), run sentence by sentence with a noise filter.	Rules found only 43% of names and 38% of addresses. NER took overall recall from 68.9% to 97.3%.
5	Layer 3: local LLM	Qwen3-4B through Ollama, structured JSON output, thinking off, opt-in as "Deep scan".	Catches the last few unusual items. Slow (~4 s) and over-flags, so it stays optional.
6	Files	OCR with Tesseract, PDF text layers with pypdfium2, PowerPoint with python-pptx, redaction drawn with Pillow.	People upload documents, not just text.
7	QR codes, barcodes, faces	zxing-cpp + OpenCV locate codes; YuNet locates faces. All are blurred and never decoded.	A boarding pass or Aadhaar QR contains all the personal data. We don't need to read it, only hide it.
8	Extension	WXT (Manifest V3, Chrome + Firefox): paste/drop/file-picker interception, review cards, vault, masking, restoring.	The user stays in their normal chatbot and keeps the final say.
9	Manual hiding	Click to hide, typed phrases, drawn boxes, "always hide" list.	No detector is perfect, and people decide for themselves what is private.
10	Cloud fallback, removed	A Groq API relay was built, then deliberately removed.	Any cloud path breaks the promise that nothing leaves the laptop, and needs a secret key to protect.
11	In-browser fallback	local-detect.ts: rules and checksums ported to TypeScript, identical results to Python on 157 documents.	If the engine is off, the user is still protected instead of failing open.
12	Evaluation	150 synthetic documents with decoys, plus 15 automated tests.	Real numbers for the judges, and proof that each layer earns its place.
13	UI redesign	New review cards with "Review" and "What the chatbot gets" tabs, hover labels, redesigned popup.	A privacy tool has to look trustworthy and be quick to understand.
14	One-step installer	install.sh sets up everything and starts the engine automatically at login.	Any laptop goes from zero to working in one command.

Problems we hit and how we solved them

Problem	Fix
IndicNER is a gated model on Hugging Face (needs an access agreement)	Automatic fallback to the open Davlan XLM-RoBERTa WikiANN model
NER missed names in long text	Run NER sentence by sentence; join word pieces; fix ORG tags that follow an honorific ("Dr. X")
NER tagged form labels and headings as places or orgs	A _keep filter: per-type score thresholds, label-word and common-word lists
"Dear Mr. Sharma" captured "Mr"	Caught by tests; the pattern now allows an optional honorific
Transaction numbers, invoice codes and Luhn-valid account numbers looked like cards/Aadhaar	Exclude-context rules (TXN, INV, order, ref) and "not a card near account context"
Evaluation looked too good because templates always had cues	Added cue-free prose documents to the dataset
A user's "always hide" word split a detected address	Merge absorbs user terms already covered by a larger finding
OpenCV 5 dropped the old face cascade	Switched fully to the YuNet face model
Words typed in the file review were silently not hidden when OCR spaced them differently ("Seat12A")	Loose matching across spaces/punctuation, plus live "found / not found" marks on the preview
QR/face labels and badge text were hard to read on the dark theme	Lighter badge text in dark mode; grey instead of near-black for codes and faces
The browser's highlight API didn't paint from the extension	Replaced with a small "restored" pill; Shadow DOM made closed

3. Architecture: the big picture

Kavach has two halves that talk only to each other, and only on this laptop.



Why it is split this way

- **The extension** is the only thing that can see what you paste on a website, so it does the catching, the review UI, the masking and the restoring.
- **The engine** does the heavy work (AI models, OCR, PDF rendering), which needs Python and native libraries that can't run well inside a browser.
- **Only the background worker talks to the network**, and only to 127.0.0.1. Content scripts run inside the chatbot's page, so we keep network access out of them. That is one place to audit.
- **Fail safe, not fail open:** if the engine is off, the background worker runs the same rules in the browser, and the review card shows an "offline" badge.

The engine endpoints

Endpoint	Input	Output	Used for
POST /detect	text, deep (bool), terms (always-hide words)	entities (type, start, end, value, score, source), timings, mode	Pasted text
POST /analyze-file	file (max 25 MB), terms	job_id, kind, pages with preview image and items with boxes	Step 1 of files: find everything, show the user
POST /render-redacted	job_id, chosen item ids, placeholders, custom boxes, terms	the redacted file (base64), mime type, file name	Step 2 of files: produce the safe file
POST /match-term	job_id, a word the user typed	how many times it occurs, and its boxes on each preview	Marks typed words on the file previews before uploading
GET /health, /info	nothing	which models are loaded and their names	Popup status, badges

Files use two steps so the user can review between them. The original file is held in memory in a job store for at most 15 minutes, then dropped.

4. Detection in depth

Every piece of text, whether pasted or read out of a file, goes through the same pipeline:

```
text --> normalize --> Layer 1: rules + checksums + context
                  --> Layer 2: NER model            (names, places, orgs)
                  --> Layer 3: Qwen3-4B (only if Deep scan is on)
                  --> user "always hide" terms
                  --> merge (resolve overlaps by priority) --> final entities
```

4.1 Normalize

Indian documents often use Devanagari or other Indic digits (e.g. Hindi digits for 1, 2, 3). normalize.py maps every Indic digit to its ASCII digit **while keeping the text exactly the same length**. Positions found in the normalized text therefore point at the same characters in the original text, so masking stays exact. A Hindi-digit Aadhaar number is caught just like an English one.

4.2 Layer 1: rules, checksums and context

rules.py defines each kind of ID as a **Rule**: a regular expression, an optional **validator** (checksum), a score, and optional **context** words that must or must not appear nearby. Boundary guards make sure a match isn't part of a longer number or word.

Type	Pattern (simplified)	Validator / extra check
AADHAAR	12 digits, first digit 2-9, optional spaces or dashes	Verhoeff checksum
VID (Aadhaar virtual ID)	16 digits	Verhoeff checksum
PAN	5 letters, 4 digits, 1 letter	4th letter must be a valid holder type (P, C, H, F, A, T, B, L, J, G)
GSTIN	2-digit state + PAN + entity char + Z + check char	Mod-36 checksum, valid state code
IFSC	4 letters + 0 + 6 letters/digits	Format check (5th char is always 0)
CARD	13-19 digits	Luhn checksum; not counted as a card near account-number context
UPI	name@handle	Must not be part of an email address
EMAIL	standard email shape	-
PHONE	Indian mobile, optional +91 / 0, starts 6-9	Excludes timestamps and IDs by context
BANK_ACCOUNT	9-18 digits	Needs account context nearby ("A/c", "account")
DOB	dd/mm/yyyy or "5th March 1990"	Valid day and month
PINCODE	6 digits, first 1-9	Needs address context or a place name before it
PASSPORT, VOTER_ID, DL, VEHICLE	Indian formats	Context words raise or lower confidence

The checksums, explained simply

- **Verhoeff (Aadhaar):** the last digit of an Aadhaar is calculated from the other 11 using three small lookup tables (based on the symmetry group of a pentagon). Kavach recomputes it. If it doesn't match, the number isn't an Aadhaar. This catches every single-digit typo and every swap of two neighbouring

digits, which is why random 12-digit order numbers are rejected.

- **Luhn (cards):** from the right, double every second digit (subtract 9 if over 9), add everything up. A real card number gives a total divisible by 10.
- **PAN 4th letter:** it encodes who holds the PAN: P = person, C = company, H = Hindu undivided family, F = firm, and so on. A random 10-character code usually fails this.
- **GSTIN mod-36:** the 14th character must be Z, characters 3-12 must be a valid PAN, and the 15th character is a check character computed from the first 14 using base-36 arithmetic. The first two digits must also be a real state code.

Context rules

- **Required context:** a bare 9-18 digit number is only a bank account if words like "account" or "A/c" are nearby.
- **Exclude context:** numbers next to "TXN", "INV-", "order", "ref" or "ticket" are not treated as Aadhaar or card numbers.
- **Labelled fields:** "Name:", "Passenger:", "Father's name:", "Address:" and similar labels mark what follows as PII.
- **Name cues:** "Dear Mr. X", "Shri X", "Smt. X", "Dr. X" mark X as a person.

Why rules come first: they are instant, explainable and, with checksums, essentially never wrong. On our test set rules alone made **zero false alarms**, even with deliberate look-alike decoys.

4.3 Layer 2: the NER model

NER (named-entity recognition) is a small transformer model trained to label words as PERSON, LOCATION or ORG. It finds names that have no label in front of them, e.g. *"I met Ananya Kulkarni near Shivaji Nagar yesterday."*

- **Model choice:** ner.py first tries **ai4bharat/IndicNER** (trained for 11 Indian languages). That model is gated behind a Hugging Face access agreement, so if it isn't available it automatically uses **Davlan/xlm-roberta-base-wikiann-ner**, which is multilingual (it covers Hindi and Marathi) and is what runs today.
- **Sentence by sentence:** the text is split on . ! ? and the Devanagari full stop, with care not to split after "Mr." or "Dr.". Long sentences are cut to about 400 characters so the model never truncates.
- **Word repair:** the model works on word pieces. Kavach joins pieces back into whole words and expands each hit to full word boundaries.
- **Noise filter (_keep):** minimum confidence per type (PERSON 0.5, LOCATION 0.7, ORG 0.9). Form labels, headings and common words are dropped. A word after an honorific that the model called ORG is relabelled PERSON.
- **Loading:** the model loads in a background thread when the engine starts, so the server answers immediately and /health reports when NER is ready.

4.4 Layer 3: the local LLM (optional "Deep scan")

When the user turns on Deep scan, llm.py asks **Qwen3-4B** running in Ollama on the same laptop to list any remaining personal information.

- Text is sent in chunks of up to 3,000 characters, with a 25-second total time limit.
- Ollama's native API is used with **think=false** (no slow hidden reasoning) and a **JSON schema**, so the model can only answer with a list of {type, text} objects from an allowed set (PERSON, ADDRESS,

MEDICAL, ORG, LOCATION, DOB, OTHER).

- **The model never gives positions.** Kavach searches for each phrase in the original text itself. A phrase that isn't in the text is discarded, so the model cannot invent PII or cause wrong masking.
- LLM findings have the lowest priority in the merge, so they can never override a checksum or the user.

4.5 Merge: deciding between overlapping findings

Different layers often find the same or overlapping spans. merge.py ranks every candidate and keeps the best non-overlapping set.

Source	Priority	Note
user (always-hide terms, manual hides)	6	The user always wins
ADDRESS (any source)	4	Addresses are long and usually contain smaller hits, so they win
rule	3 (+1 if a checksum validated it)	Proven IDs are top of the automatic results
vision (faces, codes)	3	Image regions
ner	2	
llm	1	Suggestions only

- Ties are broken by length (longer wins) and score.
- A user term inside a larger finding (e.g. "Kothrud" inside a detected address) is absorbed rather than splitting the address.
- An allowlist stops common words being masked: "India", government bodies (UIDAI, RBI, Income Tax), big bank names, "Aadhaar"/"PAN" themselves, and chatbot names.

5. Files in depth

5.1 The idea: text with positions

To black out text in an image we need to know **where on the page every character is**. page.py defines **PageText**: the page's text plus one box per character. After detection finds characters 120-134, boxes_for_span() turns that into one rectangle per line of text, with a little padding.

5.2 Images (ocr.py)

- Tesseract OCR reads the image in English, Hindi and Marathi (eng+hin+mar). Words below 30% confidence are dropped.
- Each word's box is spread across its characters to build PageText.
- Detection runs on the OCR text exactly as for pasted text.

5.3 PDFs (pdf.py)

- pypdfium2 (Google's PDFium engine) renders each page at 170 DPI, up to 20 pages.
- If the page has a real text layer (20+ characters), Kavach uses it directly, with exact character boxes and no OCR errors.
- If not (a scan), the page image goes through OCR.
- **Output is rebuilt from the redacted page images.** A PDF can hide text under a black rectangle; an image-only PDF has nothing underneath, so a redaction can't be undone.

5.4 PowerPoint (pptx_redact.py)

- Walks every slide, shape, group, table cell and notes page in a fixed order, so analysis and rendering line up exactly.
- Text is replaced in place, so formatting is kept: "Rahul" becomes "[PERSON_1]".
- Pictures inside slides are OCR'd and redacted like images, then swapped back into the deck.
- Document metadata (author, last modified by, etc.) is cleared.

5.5 QR codes, barcodes and faces (visual.py)

- **Codes:** zxing-cpp finds QR, PDF417 (used on boarding passes), Aztec, DataMatrix and 1-D barcodes. OpenCV's QR detector is a backup. Kavach only uses their **positions**; it never decodes them.
- **Faces:** YuNet, a tiny face-detection model from the OpenCV model zoo.
- Both are padded and **blurred heavily** (pixelated and blurred) when rendering. We checked that the codes can't be read after redaction.

Why blur instead of decode? Decoding would mean parsing every QR format and deciding which fields are private. Blurring is simpler, faster, and safer: whatever is inside, nobody can read it.

5.6 Rendering (render.py)

- Each chosen text item becomes a black box labelled with its placeholder, e.g. [AADHAAR_1], so the chatbot still knows what kind of thing was there.
- Manual boxes the user drew are filled black with the label [HIDDEN].
- Images go back in their original format; PDFs are rebuilt; PPTX is saved with the new text and pictures.

6. The browser extension in depth

Built with **WXT** (a framework for Manifest V3 extensions) in TypeScript. One codebase builds for Chrome, Edge, Brave and Firefox.

6.1 Content script (entrypoints/content.ts)

- Runs on the chatbot sites. It adds listeners for **paste**, **drop** and **file-input change** in the **capture phase**, which runs before the website's own handlers, and stops the original event.
- **Text:** sends the text for detection, shows the review card, then re-inserts the masked text as a new paste. Events Kavach creates itself are remembered in a WeakSet called ours, so it doesn't catch its own paste.
- **Files:** always go through the review card (preview with boxes). After the user confirms, the redacted file replaces the original in the drop or file input.
- Modes: if nothing is found, the paste goes through untouched with a small toast; if the engine is down, the in-browser rules are used.

6.2 Background worker (entrypoints/background.ts)

- The **only** part that makes network requests, and only to 127.0.0.1:8000.
- detectWithFallback(): try the engine first; if it doesn't answer, run local-detect.ts in the browser and mark the result "offline".
- Owns the **vault**: the map from placeholders to real values.

6.3 Masking and the vault (lib/mask.ts, lib/vault.ts)

- Each distinct value gets a numbered placeholder per type: the same name always maps to the same [PERSON_1] within the session.
- The vault is kept in **chrome.storage.session**: in memory only, never written to disk, cleared when the browser closes. The popup's "Forget everything" button clears it immediately.

6.4 Restoring replies (lib/rehydrate.ts)

- A MutationObserver watches the chat as replies stream in.
- When a text node contains a known placeholder, it is swapped back to the real value **on screen only**. It changes text nodes only, never HTML, so it can't break the page or inject anything.
- A small pill lets the user toggle between restored values and placeholders.

6.5 The review card UI (components/ui.ts, styles.ts)

- Drawn inside a **closed Shadow DOM**: the chatbot page's scripts can't read it or restyle it.
- Two tabs: **Review** (the text with coloured highlights; hover shows the type; click to untick or to hide more) and **What the chatbot gets** (the exact masked text that will be sent).
- Badge shows "local" (engine) or "offline" (in-browser rules). Keyboard: Enter to send safely, Esc to cancel.
- For files: page previews with boxes, item list with checkboxes, and a drawing tool for manual boxes.

6.6 Popup (entrypoints/popup/)

- Engine status, and which models are loaded (NER, Deep scan, OCR languages, face detector).
- Count of items protected so far, by type.
- Switches: Protection on/off, Deep scan, Restore replies.
- "Always hide" list, and "Forget everything" (clears the vault and stats).

6.7 In-browser fallback (lib/local-detect.ts)

A TypeScript port of the rules and checksums (Verhoeff, Luhn, PAN, GSTIN, IFSC, context rules). We checked it gives **identical results to the Python rules on 157 documents**. It has no NER, so name recall is lower offline, but every checksum-backed ID is still caught.

7. Manual hiding

Detection can miss things, and only the user knows that a project codename or a relative's nickname is private. So Kavach gives four ways to hide anything:

Method	Where	Result
Click a word or select text	Review card (text)	Becomes [CUSTOM_n], hidden like any other item
Type a phrase	Review card (text and files)	Every occurrence is hidden. In files it is marked on the preview at once with a count ("2 found"), or flagged "not found"
Draw a box	File preview	That area is filled black with [HIDDEN]
Always-hide list	Popup, or "always hide" from the card	Hidden automatically in every future paste and file

User choices have the highest merge priority (6), so they always win. Always-hide terms are sent to the engine as terms (up to 200) and matched there, or matched in the browser when offline.

Typed words survive OCR quirks

OCR doesn't always read text exactly as printed: "Seat 12A" can come out as "Seat12A", and "SF 657 · 24 OCT" as "SF 657 - 24 OCT". So a typed word is matched loosely: its words, and the letters and digits inside a word, may be joined by any spaces or punctuation, or by nothing. Matching is still case-insensitive and whole-word ("light" never matches inside "Flight"). The engine and the in-browser fallback use the same rule.

In the file review card, each typed word is sent to `/match-term`. Its matches appear on the preview as dashed boxes, and its chip says how many were found. If nothing matched, the chip turns orange and says "not found", so the user knows to drag a box over it instead. Nothing is ever hidden silently or skipped silently.

8. Security and privacy design

Decision	What it protects against
Engine binds to 127.0.0.1 only	Nobody on the Wi-Fi or internet can reach it
Extension's network access limited to http://127.0.0.1:8000	The extension can't send data anywhere else, even by mistake
Only the background worker makes requests	One small, auditable network path; nothing from the page's context
No cloud API anywhere (Groq fallback removed)	No third party ever sees raw data; no secret keys to leak
Vault in storage.session	Real values are never written to disk and vanish when the browser closes
Closed Shadow DOM	Chatbot page scripts can't read the review card, which shows the real values
Restore edits text nodes only	No HTML injection; placeholders are swapped for plain text
PDFs rebuilt from images; PPTX metadata scrubbed	No hidden text layer or author name leaks
QR codes and barcodes blurred, never decoded	Encoded data can't be read and isn't processed
Jobs expire after 15 minutes; 25 MB file limit	Uploaded originals don't linger in memory
Fails safe when the engine is off	In-browser rules still catch IDs; the user is told "offline"
Human review before anything is sent	The final decision is always the user's
Only synthetic data in demos and tests	No real person's data in the repo

One honest caveat: restored values are drawn on your screen, so the chatbot's own page scripts could in theory read the restored text. The chatbot's *servers* never receive it, and the restore can be switched off in the popup.

9. The one-step installer

Goal: someone with a fresh Mac or Linux laptop runs one command and everything works, including after a restart.

```
./install.sh                # everything; engine starts at every login
./install.sh --with-llm     # also Ollama + Qwen3-4B for Deep scan (~2.5 GB)
./install.sh --no-autostart # don't register autostart; use ./run.sh instead
./uninstall.sh              # remove the autostart
```

Step	What it does
1. System tools	Installs Tesseract with language packs and Node (Homebrew on macOS, apt on Linux); checks Python 3.10+. With --with-llm, installs Ollama and starts it as a login service.
2. Engine	Creates a Python virtual environment, installs requirements, downloads the YuNet face model, pre-downloads the NER model so the first paste is fast. With --with-llm, pulls qwen3:4b.
3. Extension	npm install, builds the Chrome and Firefox versions, and copies zips to dist/kavach-chrome.zip and dist/kavach-firefox.zip.
4. Autostart	macOS: a LaunchAgent (com.kavach.engine) with RunAtLoad and KeepAlive, so it starts at login and restarts if it crashes. Linux: a systemd user service with Restart=always. Refuses if port 8000 is already in use.
5. Check	Waits for /health to answer, then prints how to load the extension (Load unpacked → extension/.output/chrome-mv3).

We tested it on a real laptop: about 14 seconds when packages are cached; the engine restarted by itself after being force-killed; the browser tests passed against the auto-started engine.

10. Evaluation: how we measured it

The dataset

- **150 synthetic documents** generated with Faker (fixed seed 42, so results are repeatable): ID-card text, bank letters, invoices, medical notes, emails, Hindi/Marathi-mixed text and cue-free prose.
- Every real-looking ID is generated with a valid checksum, so it should be found.
- **Decoys** that must *not* be flagged: order IDs that fail the Aadhaar checksum, invoice and transaction numbers, timestamps.
- **Recall** = share of real PII items found. **Precision** = share of flagged items that really were PII.

Results

Setup	Docs	Recall	Precision	False alarms	ms / doc
Rules + checksums	150	68.9%	100.0%	0	0
Rules + NER (default)	150	97.3%	100.0%	0	58
Rules + NER (same 30-doc subset)	30	97.2%	100.0%	0	55
Rules + NER + Qwen3-4B	30	99.3%	86.0%	23	3,969

Recall by type

Type	Rules	Rules + NER	+ Qwen3
AADHAAR, CARD, DOB, EMAIL, GSTIN, IFSC, PAN, PHONE, UPI	100%	100%	100%
BANK_ACCOUNT	98%	98%	100%
PERSON	43%	96%	100%
ADDRESS	38%	90%	91%

What the numbers tell us

- **Checksums make rules precise:** zero false alarms despite many look-alike numbers.
- **Rules can't find names and addresses:** only 43% and 38%, because most names have no label.
- **The small open-weight NER model is what makes Kavach work:** +28 points of recall, no loss of precision, 58 ms on a CPU.
- **The 4B LLM adds the last 2 points but costs 70x the time and 23 false alarms**, so it is opt-in, not default.

Automated tests

15 pytest tests cover the validators (Verhoeff, Luhn, PAN, GSTIN, IFSC), rule edge cases (decoys, "Dear Mr.", UPI before a full stop, Indic digits), merge behaviour and user terms (including OCR-spacing tolerance). End-to-end browser tests (Playwright) checked paste, mask, restore, a boarding pass (9 items), a scanned PDF (13), a PPTX (21), manual boxes and the offline mode.

11. Why we chose each technology

Choice	Why	What we didn't use, and why
Browser extension	Works inside the chatbot you already use, at the exact moment of the leak	Desktop proxy: needs to intercept HTTPS, invasive. Separate website: extra step people skip
Local Python engine (FastAPI + Uvicorn + Pydantic)	All the AI/OCR/PDF tools are Python; FastAPI is fast and type-checked	Everything in the browser (WASM): too slow and heavy for NER + OCR + PDF
Rules + checksums first	Indian IDs have check digits: proof beats prediction; instant	AI for everything: slower, and guesses at what maths can prove
XLM-RoBERTa / IndicNER	Multilingual incl. Hindi/Marathi, ~280M params, CPU-fast	Large LLM for all detection: ~70x slower, less precise
Qwen3-4B (optional)	Strong small open model, good at structured JSON, Apache-2.0	8B+ models: too heavy for a student laptop
Ollama	One-command install and model serving, MIT	Hand-built llama.cpp: harder setup for others
Tesseract OCR	Open source, offline, Hindi + Marathi packs	Cloud OCR: would send the document out
pypdfium2	Exact character boxes from PDF text, fast rendering, permissive license	PyMuPDF: AGPL license would force the project's license
Pillow	Drawing boxes, blurring, writing images/PDFs	img2pdf: LGPL; not needed
zxing-cpp + OpenCV	Find every common barcode type, quickly	Decoding QR contents: unnecessary and riskier
YuNet	Tiny, accurate face detector	Old Haar cascades: removed in OpenCV 5, less accurate
python-pptx	Edit text in place, keeping formatting	Converting to images: would lose editable slides
WXT + TypeScript	One codebase for Chrome and Firefox, typed, fast builds	Hand-written per-browser manifests
Closed Shadow DOM	Page scripts can't read our UI	Normal DOM overlay: readable by the chatbot's scripts
storage.session vault	In memory only, cleared on browser close	localStorage: real values would persist on disk
No cloud fallback	Keeps the promise simple and true	Groq relay: built, then removed on purpose

12. Why this is a strong use of light open-weight models

This is the heart of the Track 1 story. Kavach doesn't just *use* open models; it is built around the fact that small, open, local models are the **only** way this product can keep its promise.

1. Privacy is only possible because the models are open and light

A PII tool that sends your Aadhaar to a cloud API in order to *find* your Aadhaar defeats itself. Open weights that run on a normal laptop CPU, with no GPU and no internet, are what make "nothing leaves your device" true rather than a policy promise.

2. The right-sized model for each job, in a cascade

```
cost / time  --->
[ maths: checksums ]  ->  [ small model: NER ~280M ]  ->  [ medium model: Qwen3-4B, opt-in ]
   ~0 ms, 100% precise      58 ms, finds names/places        ~4 s, finds the last few
```

Each layer only does what the cheaper layer can't. The evaluation proves the design: the small NER model adds **+28 points of recall for 58 ms** with no loss of precision; the 4B model adds only 2 more points at 70x the cost and brings false alarms. So the small model is the default and the bigger one is a button.

3. The LLM is kept on a short leash

- It only **suggests phrases**; it never decides positions.
- Every phrase is searched for in the original text; anything not found is dropped, so it can't invent PII.
- Output is forced into a JSON schema with a fixed list of types; thinking is turned off for speed.
- It has the lowest merge priority, so it can never override a checksum-proven ID or the user's choice.

4. AI where it's needed, maths where it's possible

IDs are handled by checksums because they can be proven. Names, places and addresses are handled by models because only language understanding can find them. Using a model where a checksum works would be slower and less reliable.

5. Small footprint, zero running cost

Component	Approximate size	Needs
NER model (XLM-RoBERTa base)	~1.1 GB on disk, ~280M parameters	CPU
Tesseract + eng/hin/mar data	~100 MB	CPU
YuNet face model	~0.3 MB	CPU
Qwen3-4B (optional, quantized)	~2.5 GB	CPU or Apple GPU; 8 GB+ RAM

No API keys, no per-request cost, and no internet after setup. Any student or office laptop can run it, which matters for Indian users who can't pay for enterprise data-loss-prevention tools.

6. Fully open and auditable

Every model is open-weight, every library is permissively licensed, and the repository is MIT. A privacy tool people can't inspect asks for blind trust; Kavach can be read line by line.

7. Built for India specifically

Indian ID formats with their real checksums, Indic digit normalization, Hindi and Marathi OCR, a multilingual NER model (with IndicNER support built in), and handling of Indian documents like boarding passes, Aadhaar cards and GST invoices.

13. File-by-file reference

Engine (engine/)

File	What it does
server.py	FastAPI app: /health, /info, /detect, /analyze-file, /render-redacted, /match-term, /test. Loads NER in a background thread at startup. 25 MB file limit.
schemas.py	Pydantic models for every request and response: Entity, DetectRequest/Response, FileItem, FilePage, AnalyzeFileResponse, RenderRequest/Response, HealthResponse. The contract between engine and extension.
detect/__init__.py	detect(text, deep, terms): runs normalize, rules, user terms, NER, LLM (if deep) and merge; returns entities and timings.
detect/normalize.py	Maps Indic digits to ASCII without changing text length.
detect/validators.py	Verhoeff, Luhn, PAN holder type, GSTIN mod-36, IFSC checks.
detect/rules.py	The Rule definitions, context and exclude rules, labelled fields, name cues, and find_terms for always-hide words.
detect/ner.py	Loads IndicNER or falls back to XLM-RoBERTa; sentence splitting, word repair, the _keep noise filter.
detect/llm.py	Calls Qwen3-4B in Ollama with a JSON schema and think=false; locates phrases in the original text.
detect/merge.py	Priority-based overlap resolution, user-term absorption, allowlist.
docs/page.py	PageText (text + per-character boxes) and boxes_for_span.
docs/ocr.py	Tesseract OCR into PageText; language detection; image opening.
docs/pdf.py	pypdfium2 rendering at 170 DPI, text layer or OCR fallback, 20-page limit.
docs/visual.py	find_codes (zxing-cpp + OpenCV) and find_faces (YuNet).
docs/render.py	Black labelled boxes, blurring, image/PDF output, previews.
docs/pptx_redact.py	Walks slides in a fixed order, replaces text, swaps redacted pictures, scrubs metadata.
docs/jobs.py	In-memory job store for the two-step file flow, 15-minute expiry.
docs/router.py	Ties it together: analyze a file into pages and items; match typed words for the preview; render the chosen items, custom boxes and terms ([HIDDEN]).
dev/test.html	A fake chatbot page served at /test, to try Kavach offline.
eval/make_samples.py	Generates demo files (ID card, boarding pass with PDF417/Aztec/QR, scanned PDF, PPTX) with fake data.
eval/evaluate.py	Builds the 150-doc synthetic dataset and computes recall, precision and timing.
tests/	15 pytest tests for validators, rules, merge and terms.
requirements.txt	Python dependencies.

Extension (extension/)

File	What it does
entrypoints/content.ts	Catches paste, drop and file-input events; runs the review flow; re-sends masked text or redacted files.

File	What it does
entrypoints/background.ts	All network calls (engine only), in-browser fallback, vault.
entrypoints/popup/*	Popup page: status, models, stats, switches, always-hide list, forget button.
components/ui.ts	Review cards, file previews, drawing tool, toasts, restore pill (closed Shadow DOM).
components/styles.ts	Styles for the cards, light and dark mode.
lib/types.ts	Shared TypeScript types (Entity, Settings, responses).
lib/messages.ts	Typed messages between content script, popup and background.
lib/engine.ts	HTTP client for the local engine.
lib/local-detect.ts	Rules and checksums in TypeScript for offline mode.
lib/mask.ts	Builds placeholders and the masked text.
lib/vault.ts	Placeholder-to-value map in storage.session.
lib/rehydrate.ts	Restores placeholders in replies on screen (MutationObserver).
lib/files.ts	Reading files, rebuilding File objects, swapping them into drops/inputs.
lib/settings.ts	Settings, stats and always-hide terms.
lib/colors.ts	Colour and label per PII type.
wxt.config.ts	Manifest: sites, permissions (only 127.0.0.1:8000), Firefox settings.

Top level

File	What it does
install.sh / uninstall.sh	One-step setup with autostart, and its undo.
run.sh	Start the engine by hand (development).
README.md	Install, usage, models and licenses, limitations.
LICENSE	MIT.
docs/CODEBASE_GUIDE.txt, docs/eval-results.md, docs/demo-samples/	Team guide, evaluation output, demo files.

14. Models, tools and licenses

Open-weight models (all run locally)

Model	Used for	License
ai4bharat/IndicNER	Names, places, orgs in 11 Indian languages (primary, when access granted)	MIT
Davlan/xlm-roberta-base-wikiann-ner	Multilingual NER (active fallback)	Not stated on card; base XLM-R is MIT
Qwen3-4B via Ollama	Optional Deep scan	Apache-2.0
Tesseract LSTM models (eng, hin, mar)	OCR for images and scans	Apache-2.0
YuNet (OpenCV Zoo)	Face detection for blurring	MIT

Libraries and tools

Tool	Purpose	License
FastAPI, Uvicorn, Pydantic	Local engine API	MIT / BSD-3 / MIT
Transformers, PyTorch	Running NER	Apache-2.0 / BSD-3
httpx	Calling Ollama	BSD-3
pypdfium2 (PDFium)	PDF text and rendering	Apache-2.0 / BSD-3
pytesseract	Tesseract wrapper	Apache-2.0
Pillow	Redaction drawing, PDF writing	MIT-CMU
OpenCV (headless)	YuNet runtime, backup QR detection	Apache-2.0
zxing-cpp	Locating QR codes and barcodes	Apache-2.0
python-pptx	PowerPoint editing	MIT
Faker, reportlab, pytest	Synthetic data, demo PDFs, tests	MIT / BSD / MIT
Ollama	Local LLM runtime	MIT
WXT, Vite, TypeScript	Extension build	MIT / MIT / Apache-2.0
Playwright	End-to-end browser tests (development)	Apache-2.0

PyMuPDF (AGPL) and img2pdf (LGPL) were avoided on purpose so the whole project stays permissively licensed. Parts of the code were written with Claude Code (Anthropic); the README says so, and the team can explain every module.

15. Honest limitations and what comes next

Limitations

- Files attached through cloud connectors (Google Drive, OneDrive) never pass through the browser, so Kavach can't see them.
- Handwriting and blurry photos OCR poorly; anything OCR misses can't be redacted. That is why the review card exists.
- IndicNER needs a Hugging Face access agreement; until then the multilingual XLM-RoBERTa model is used.
- Restored values are drawn on screen, so the chatbot page's own scripts could read them (its servers never receive them).
- Typed text (not pasted) isn't intercepted yet.
- The evaluation uses synthetic data, because we deliberately used no real personal data.
- The installer covers macOS and Linux; Windows works through WSL.

What comes next

- Enable IndicNER by default and fine-tune on Indian-name data.
- Right-click "mask selection" and protection for typed text.
- DOCX and XLSX support; more OCR languages (Tamil, Telugu, Bengali, Gujarati...).
- A signed, packaged installer for Windows; a Chrome Web Store / Firefox Add-ons listing.
- Organisation policies (e.g. always block Aadhaar) for schools and companies.

16. 4-minute pitch script

Time	Say	Show
0:00-0:30	"How many of you have pasted a bill, a form or an ID into ChatGPT? Once you press paste, your Aadhaar, PAN and bank details are on someone else's servers, forever. Existing tools don't know Indian IDs, and most send your data to their own cloud to check it."	Title slide
0:30-1:00	"Kavach is a browser extension with a small engine that runs on your own laptop. It catches the paste before the website sees it, finds the personal data locally, and sends the chatbot placeholders like PERSON_1. When the chatbot answers, you see your real data again, on your screen only."	Architecture diagram
1:00-2:15	Live demo: paste a bank letter into ChatGPT, review card appears, switch to "What the chatbot gets", send, reply shows restored name. Then drop the boarding pass: names boxed, PDF417 and QR codes blurred.	Live chatbot
2:15-3:00	"Under the hood: three layers. Checksums like Verhoeff for Aadhaar give 100% precision. A small open-weight NER model takes recall from 69% to 97% in 58 milliseconds on a CPU. An optional Qwen3-4B deep scan finds the last 2%."	Eval table
3:00-3:40	"Why light open-weight models? Because a privacy tool can't send your data to the cloud to protect it. Small open models running offline are the only way to keep that promise. And everything is open: models, libraries, MIT repo."	Models + licenses
3:40-4:00	"One command installs it. Nothing leaves your laptop. Kavach: armour for your data in the age of AI. Thank you."	Install slide

Demo tip: have the engine already running (it autostarts), the boarding pass and bank letter ready in docs/demo-samples/, and the offline test page (http://127.0.0.1:8000/test) open as a backup if the Wi-Fi fails.

17. Likely judge questions and answers

Q: Where exactly is the AI in this project?

The NER model (open-weight XLM-RoBERTa, or IndicNER) finds names, places and organisations; it is what lifts recall from 69% to 97%. Optionally Qwen3-4B finds remaining items. Tesseract's LSTM OCR models read images, and YuNet finds faces. All run locally.

Q: Why not just use GPT or Gemini to detect PII?

Then the PII has already been sent to the cloud, which is the leak we're preventing. Our numbers also show a small model is faster and more precise for this task.

Q: What if the engine isn't running?

The extension runs the same rules and checksums in the browser and shows an "offline" badge. Every checksum-backed ID is still caught; only name detection is weaker.

Q: Can the chatbot ever see the real data?

No. It only receives placeholders. Real values live in a session-only vault in the browser and are put back on your screen only.

Q: How do you avoid false alarms on random numbers?

Checksums. A random 12-digit number has only about a 1 in 10 chance of passing Verhoeff, and context rules reject numbers next to words like TXN, invoice or order. We had zero false alarms on 150 documents with decoys.

Q: How do you handle QR codes?

We find them with zxing-cpp and OpenCV and blur them. We never decode them, so whatever is inside is hidden.

Q: How is a redacted PDF safe? Can't someone remove the black box?

We rebuild the PDF from the redacted page images, so there is no text layer under the box to recover.

Q: Does it support Indian languages?

Yes: Indic digits are normalized, OCR reads Hindi and Marathi, and the NER model is multilingual. IndicNER support for 11 languages is built in.

Q: How did you evaluate it?

150 synthetic documents with valid-checksum IDs and deliberate decoys, measuring recall, precision and time for each layer combination. Plus 15 unit tests and end-to-end browser tests.

Q: Did you use AI to write the code?

Yes, Claude Code, and the README says so, as the rules allow. We can explain every layer: the checksum maths, the merge priorities, and how pastes and uploads are intercepted.

Q: Why an extension and not an app?

The leak happens inside the browser at the moment of pasting. An extension catches it there with no change to the user's habits.

Q: Why did you remove the cloud fallback?

Any cloud path weakens the core promise and needs a secret key. A purely local design is simpler to trust and to audit.

Q: How heavy is it?

About 1 GB by default (mostly the NER model), all on CPU. Deep scan adds ~2.5 GB and is optional.

Q: What would you do with more time?

Enable IndicNER by default, add typed-text protection, DOCX/XLSX, more OCR languages, and a Windows installer.

18. Glossary

Term	Meaning
PII	Personally identifiable information: anything that identifies a person (name, Aadhaar, phone, address...)
Redaction / masking	Hiding PII. Masking replaces it with a placeholder; redaction blacks it out in a file.
Placeholder	A token like [PERSON_1] that stands in for a real value.
Vault	The in-browser map from placeholders back to real values.
Rehydrate / restore	Putting real values back on screen in the chatbot's reply.
Checksum	A digit computed from the others, used to detect typos and fake numbers (Verhoeff, Luhn, mod-36).
NER	Named-entity recognition: a model that labels words as person, place or organisation.
Open-weight model	A model whose trained weights are published, so anyone can run it locally.
LLM	Large language model (here Qwen3-4B, small by LLM standards).
OCR	Optical character recognition: reading text from images.
Recall / precision	Recall: share of real PII found. Precision: share of flagged items that really are PII.
Manifest V3	The current standard for Chrome extensions.
Content script / background worker	The extension part that runs inside the web page / the part that runs separately and can use the network.
Shadow DOM (closed)	A sealed piece of page UI that the page's own scripts can't access.
Capture phase	The first phase of a browser event, before the website's own handlers run.
LaunchAgent / systemd	macOS / Linux mechanisms that start a program at login and keep it running.